

Engenharia de Software

Requisitos

Prof. Tiago Piperno Bonetti

Material adaptado de: Marco Tulio Valente
<https://engsoftmoderna.info>



Casos de Uso

Casos de Uso

- » Documento mais detalhado de especificação de requisitos
- » Uso **não** é tão comum com métodos ágeis
- » Um ator realizando alguma operação com o sistema
- » Incluem fluxo normal e extensões
- » Extensões:
 - Exceções (ou erros)
 - Detalhamento

Transferir Valores entre Contas

Ator: Cliente do Banco

Fluxo normal:

- 1 - Autenticar Cliente
- 2 - Cliente informa agência e conta de destino da transferência
- 3 - Cliente informa valor que deseja transferir
- 4 - Cliente informa a data em que pretende realizar a operação
- 5 - Sistema efetua transferência
- 6 - Sistema pergunta se o cliente deseja realizar uma nova transferência

Extensões:

- 2a - Se conta e agência incorretas, solicitar nova conta e agência
- 3a - Se valor acima do saldo atual, solicitar novo valor
- 4a - Data informada deve ser a data atual ou no máximo um ano a frente
- 5a - Se data informada é a data atual, transferir imediatamente
- 5b - Se data informada é uma data futura, agendar transferência

Transferir Valores entre Contas

Ator: Cliente do Banco

Fluxo normal:

- 1 - Autenticar Cliente
- 2 - Cliente informa agência e conta de destino da transferência
- 3 - Cliente informa valor que deseja transferir
- 4 - Cliente informa a data em que pretende realizar a operação
- 5 - Sistema efetua transferência
- 6 - Sistema pergunta se o cliente deseja realizar uma nova transferência

Extensões:

- 2a - Se conta e agência incorretas, solicitar nova conta e agência
- 3a - Se valor acima do saldo atual, solicitar novo valor
- 4a - Data informada deve ser a data atual ou no máximo um ano a frente
- 5a - Se data informada é a data atual, transferir imediatamente
- 5b - Se data informada é uma data futura, agendar transferência

Erro (passo 2)

Transferir Valores entre Contas

Ator: Cliente do Banco

Fluxo normal:

- 1 - Autenticar Cliente
- 2 - Cliente informa agência e conta de destino da transferência
- 3 - Cliente informa valor que deseja transferir
- 4 - Cliente informa a data em que pretende realizar a operação
- 5 - Sistema efetua transferência
- 6 - Sistema pergunta se o cliente deseja realizar uma nova transferência

Extensões:

- 2a - Se conta e agência incorretas, solicitar nova conta e agência
- 3a - Se valor acima do saldo atual, solicitar novo valor
- 4a - Data informada deve ser a data atual ou no máximo um ano a frente
- 5a - Se data informada é a data atual, transferir imediatamente
- 5b - Se data informada é uma data futura, agendar transferência

Detalhamento
(passo 5)

Importante

- » Casos de uso não são "algoritmos"
- » Ainda estamos levantando requisitos:
 - Foco: entendimento e delimitação do problema
 - E não em possíveis soluções (i.e., algoritmos)

Boas práticas para casos de uso

- » As ações devem ser escritas em uma **linguagem simples e direta**.
 - Sempre que possível, use o ator principal como sujeito das ações, seguido de um verbo.
Exemplo: o cliente insere o cartão no caixa eletrônico. Se a ação for realizada pelo sistema, escreva algo como: o sistema valida o cartão inserido.
- » Casos de uso devem ser **pequenos, com poucos passos**, principalmente no fluxo normal, para facilitar o entendimento.
 - Se você estiver escrevendo um caso de uso e ele começar a ficar extenso, tente quebrá-lo em dois casos de uso menores. Outra alternativa consiste em agrupar alguns passos. Exemplo, os passos usuário informa login e usuário informa senha podem ser agrupados em usuário informa login e senha.

Boas práticas para casos de uso

- » Casos de uso **não são algoritmos** escritos em pseudo-código.
 - O nível de abstração é maior do que aquele necessário em algoritmos. Lembre-se de que os usuários do sistema devem ser capazes de ler, entender e descobrir problemas em casos de uso. Exemplo: em vez de um comando de repetição, você pode usar algo como: o cliente pesquisa o catálogo até encontrar o produto que pretende comprar.
- » Casos de uso **não** devem tratar de **aspectos tecnológicos ou de design**.
 - Exemplo: não se deve escrever algo como: o cliente pressiona o botão verde para confirmar a transferência.

Boas práticas para casos de uso

- » Evite casos de uso **muito simples**, como aqueles com apenas operações **CRUD** (Cadastrar, Recuperar, Atualizar ou Update e Deletar).
 - Por exemplo, em um sistema acadêmico não faz sentido ter casos de uso como Cadastrar Professor, Recuperar Professor, Atualizar Professor e Deletar Professor. No máximo, crie um caso de uso Gerenciar Professor e explique brevemente que ele inclui essas quatro operações.
- » Padronize o **vocabulário** adotado nos casos de uso.
 - Exemplo, evite usar o nome Cliente em um caso de uso e Usuário em outro. David Thomas e Andrew Hunt recomendam a criação de um glossário, isto é, um documento que lista os termos e vocabulário usados em um projeto.

Exemplo: Venda em Caixa de Supermercado (PDV)

Fonte: Craig Larman. Applying UML and Patterns. Pearson, 2004

Fluxo Normal

1. Cliente chega no caixa com os produtos que deseja comprar
2. Caixa inicia uma nova venda
3. Caixa identifica um produto; por exemplo, usando leitor de código de barras
4. Sistema identifica produto, registra venda, apresenta a descrição do produto e seu preço, bem como o total da compra até o momento
5. Caixa repete passos 3-4 até não haver mais produtos para registrar venda
6. Sistema apresenta total da venda
7. Caixa informa total da venda para o cliente e pede o pagamento
8. Cliente faz o pagamento e o sistema processa o pagamento
9. Sistema registra a venda como concluída e envia informações para o sistema de contabilidade e para o sistema de controle de estoques
10. Sistema gera recibo da venda
11. Caixa entrega recibo para o cliente
12. Cliente encerra a compra, levando os produtos e o seu recibo

Extensões (ou fluxos alternativos): [vamos mostrar para apenas um dos passos do fluxo normal]

7a. Pagamento em dinheiro:

1. Caixa digita o montante de dinheiro que o cliente lhe forneceu
2. Sistema informa o valor do "troco" e libera a gaveta de notas
3. Caixa deposita o dinheiro na gaveta e retorna "troco" para o cliente Cliente
4. Sistema registra e conclui pagamento com dinheiro

7b. Pagamento com cartão de crédito:

1. Cliente insere cartão na máquina de cartão de crédito
2. Sistema informa para máquina de cartão o valor da compra
3. Cliente informa senha e confirma compra
4. Sistema envia solicitação de pagamento para operadora do cartão
 - 4a. Se erro de comunicação com o sistema da operadora do cartão
 1. Sistema sinaliza erro para o Caixa
 2. Caixa solicita ao Cliente um modo alternativo de pagamento

(continua no próximo slide)

5. Sistema recebe resultado da requisição de pagamento
 - 5a. Pagamento negado
 1. Sistema avisa o Caixa
 2. Caixa solicita ao Cliente um modo alternativo de pagamento
 - 5b. Timeout na espera pelo resultado da requisição de pagamento
 1. Sistema avisa o Caixa
 2. Caixa tenta de novo ou solicita modo alternativo de pagamento
6. Sistema registra e conclui pagamento com cartão de crédito

7c. Pagamento com cheque

....

7d. Pagamento com cartão de débito

....

Diagramas de Casos de Uso

Diagramas de Casos de Uso

- » Linguagem de modelagem gráfica UML
- » Esse diagrama é um **índice gráfico** de casos de uso.
- » Ele representa os **atores** de um sistema (como pequenos bonecos) e os **casos de uso** (como elipses).
- » Dois tipos de **relacionamento**:
 - Ligando ator com caso de uso, que indicam que um ator participa de um determinado caso de uso;
 - Ligando dois casos de uso, que indicam que um caso de uso inclui ou estende outro caso de uso.

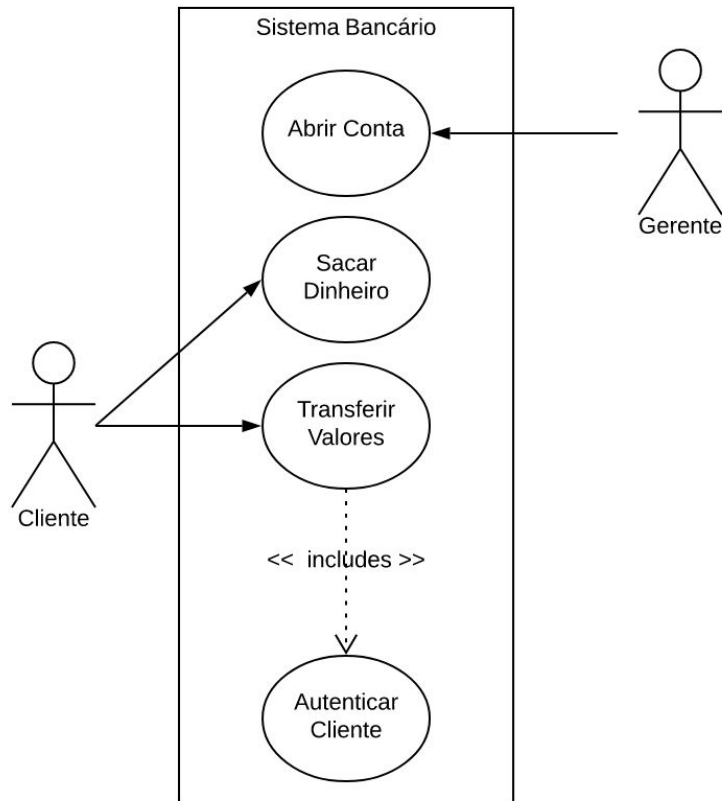
Diagramas de Casos de Uso

Dois atores: Cliente e Gerente.

- » Cliente participa dos seguintes casos de uso: Sacar Dinheiro e Transferir Valores. E Gerente é o ator principal do caso de uso Abrir Conta.

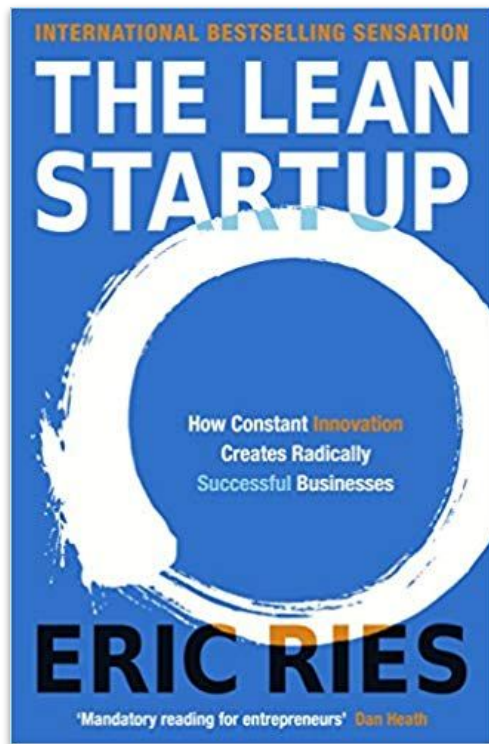
Transferir Valores inclui o caso de uso Autenticar Cliente.

Os casos de uso são representados dentro de um retângulo, que delimita as fronteiras do sistema.



Produto Mínimo Viável

Origem



2011

Produto Mínimo Viável (MVP)

- » Em uma empresa de desenvolvimento de software, o **maior desperdício** que pode existir é passar **anos levantando requisitos e implementando** um sistema que depois **não vai ser usado**.
- » Se é para um sistema falhar — por não ter sucesso, usuários ou mercado — **é melhor falhar rapidamente**.
- » Eles são mais comuns em **startups**, pois por definição elas são empresas que **operam em ambientes de grande incerteza**.

Dois tipos de sistemas

Baixo risco e usuários conhecidos

- » Exemplo: sistema de controle de bibliotecas
- » Sistema conhecido, fundamental em toda biblioteca, etc
- » Necessidade e viabilidade desse sistema são óbvias

Alto risco e sucesso incerto

- » Exemplo: loja virtual para empréstimo de livros digitais com pagamento via bitcoins
- » Sistemas típicos de startups, mas não exclusivos
- » Como risco é alto, ideia tem que ser rapidamente validada com usuários reais.

Produto Mínimo Viável (MVP)

- » Produto = já pode ser usado
- » Mínimo = menor conjunto de funcionalidades (menor custo)
- » Viável? = objetivo é obter dados e validar uma ideia

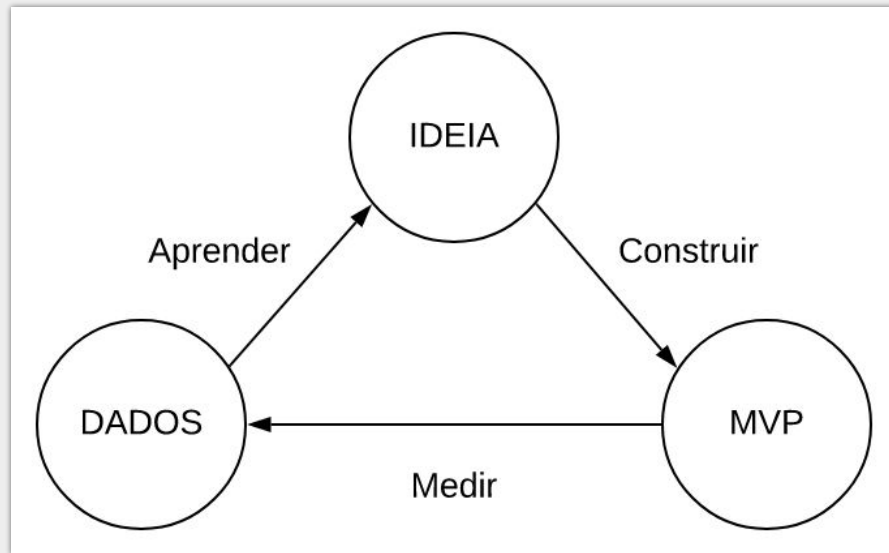
Costuma-se dizer que o objetivo de um MVP é **testar uma hipótese** de negócio.

Produto Mínimo Viável (MVP)

Lean startup propõe um **método** sistemático e científico para **construção e validação de MVPs**.

Esse método consiste em um ciclo com três passos:

- Construir;
- Medir;
- Aprender

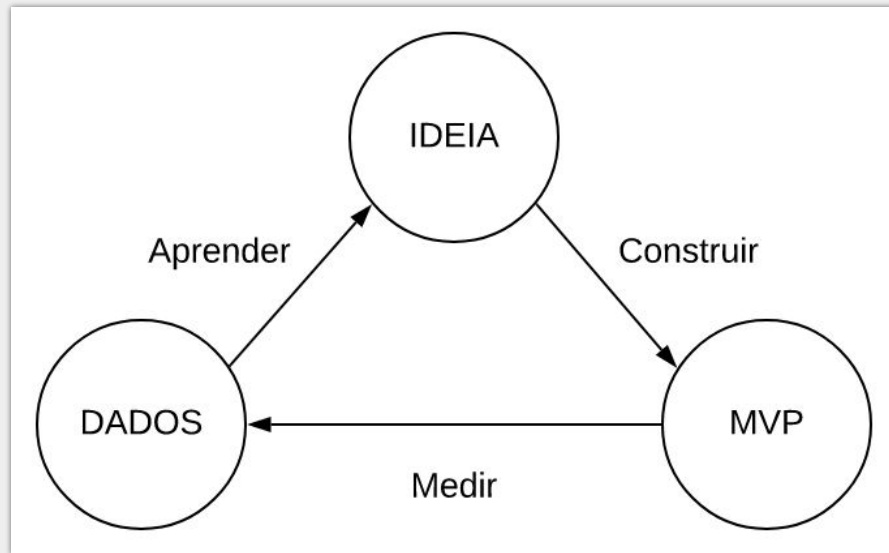


Produto Mínimo Viável (MVP)

Construir: tem-se uma ideia de produto e então implementa-se um MVP para testá-la.

Medir: o MVP é disponibilizado para uso por clientes com o intuito de coletar dados sobre a sua viabilidade.

Aprender: as métricas coletadas são analisadas e geram o que se denomina de aprendizado validado (validated learning)



Ciclo Construir-Medir-Aprender

- » Ao final desse ciclo, pode-se:
 - Realizar alguns ajustes e rodar ciclo de novo
 - Pivotar: mudar foco do produto
 - Desistir (dinheiro acabou!)
 - Deu certo: atingimos o “product marke fit” e vamos agora construir um produto mais robusto

***MVP não precisa ser um
software completo***

Zappos

- » Loja virtual de sapatos, depois adquirida pela Amazon por mais de um bilhão de dólares.
- » As pessoas vão comprar sapatos pela Internet? (em 1999)
- » MVP:
 - Sistema Web simples
 - Com fotos de sapatos de lojas físicas da cidade
 - Backend era 100% manual
- » Objetivo: apenas validar hipótese de negócio.



women's

men's

dress

casual

athletic

dress

casual

athletic

kids

Pick a category
to shop from:

Category

Register now &

Save Money

registered customers

username

password

login



Measure Your Foot

The world's largest shoe store!

WHAT WE'RE HEARING!

Welcome to Zappos.com - the shoe store! We have a selection of over 100 brands to shop from! We offer FREE SHIPPING (U.S. orders only) and NO SALES TAX.

Live Customer Service!

Mon. - Fri. 10am-6pm PST

Sat. 10am-5pm PST

Click here!



Free Shoes!



Congratulations to the November 24 winner,
Adriana Rodriguez of Oakland, CA

We will be giving away a FREE PAIR OF SHOES (up to \$150 in store credit) every Wednesday until the year 2000!

To enter become a registered user or send an email to freeshoes@zappos.com with your name and email address. A winner will be notified by email every Wednesday!

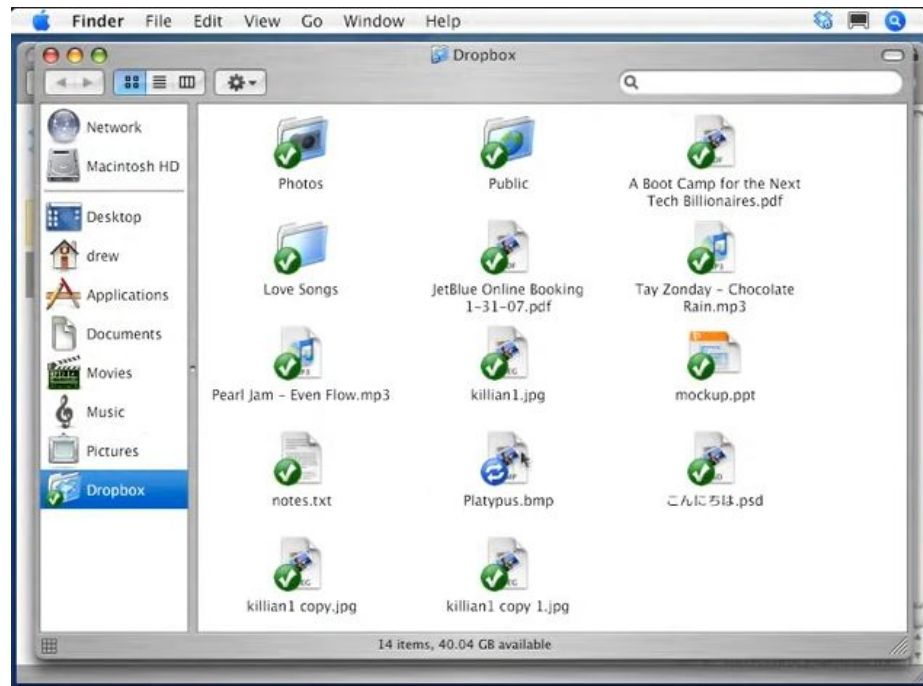
featured brands.



Zappos Special:

Free Shipping &
No Sales Tax

MVP do Dropbox (apenas um vídeo para atrair usuários para a versão beta)



<https://www.youtube.com/watch?v=7QmCUDHpNzE&t=1s>

Implementando um carro



Sem
MVP



Com
MVP

Fonte: <https://blog.nubank.com.br/mvp-o-que-e-nubank/>

MVP ≠ 1a versão de um produto



MVP é um experimento (logo, pode falhar)

» Ou seja:

- Se existe um mercado certo
- Se o cliente já te contratou e vai te pagar
- Se você tem competência para desenvolver
- Se o cliente sabe muito bem o que quer
- Logo, não existe risco e não precisamos de MVP

» Se já sabemos que vai dar certo, não é um experimento

MVP & Engenharia de Software

- » MVP não precisa usar todas as melhores práticas de ES
 - Testes de unidade, refatorações, arquitetura complexa, etc
- » Se a ideia der certo, pode-se depois reimplementar o sistema
- » Certos requisitos, principalmente NF, podem ser importantes
 - Desempenho, usabilidade, estabilidade, etc

Algumas frases sobre MVPs

- » “Se você não tiver vergonha do seu MVP, você demorou demais para lançá-lo” (Reid Hoffman).
- » “Se eu tivesse perguntado para meus clientes o que eles queriam, a resposta teria sido um cavalo mais rápido” (Henry Ford).



Construindo o Primeiro MVP

- » Em **alguns casos** isso **não é um problema**, pois os proponentes do MVP têm uma ideia precisa de suas funcionalidades e requisitos.
- » Em certos casos, a **ideia** do sistema **pode não estar clara**.
- » Nesses casos, recomenda-se **construir um protótipo**.
- » **Design Sprint** é um método para **testar e validar novos produtos por meio de protótipos**, não necessariamente de software.

Design Sprint - Principais características

» Time-box.

- Um design sprint tem a duração de cinco dias, começando na segunda-feira e terminando na sexta-feira. O objetivo é descobrir uma primeira solução para um problema rapidamente.

» Equipes pequenas e multidisciplinares.

- Deve reunir uma equipe multidisciplinar de sete pessoas.
- O objetivo é fomentar discussões.
- Devem participar representantes de todas as áreas envolvidas com o sistema.
- A equipe deve incluir um tomador de decisões, que pode ser o próprio dono da empresa.

Design Sprint - Principais características

» Objetivos e regras claras.

- No primeiro dia, entende-se e delimita-se o problema que se pretende resolver. O objetivo é garantir que, nos dias seguintes, a equipe estará focada em resolver o mesmo problema (convergência).
- No segundo dia, possíveis alternativas de solução são propostas, de forma livre (divergência).
- No terceiro dia, escolhe-se uma solução vencedora, dentre as possíveis alternativas (convergência).
 - Nessa escolha, a última palavra cabe ao tomador de decisões, isto é, um design sprint não é um processo puramente democrático.

Design Sprint - Principais características

» Objetivos e regras claras.

- No quarto dia, implementa-se um protótipo, que pode ser simplesmente um conjunto de páginas HTML estáticas, sem qualquer código ou funcionalidade.
- No último dia, testa-se o protótipo com cinco clientes reais, com cada um deles usando o sistema em sessões individuais.

Design Sprint - Principais características

dia 1



ENTENDER

- Quem são os usuários
- Quais são suas necessidades
 - Qual é o contexto
- Avaliação do concorrente
- Formulação de estratégia

2



DIVERGIR

- Imaginar
- Desenvolver múltiplas soluções
- Idear

3



DECIDIR

- Escolher a melhor ideia
- Storyboard da ideia

4



PROTOTIPAR

- Construir algo rápido para mostrar a usuários internos
- Foco na usabilidade ao invés da beleza

5



VALIDAR

- Mostrar o protótipo a usuários reais fora da organização
- Descobrir o que não funciona

Fonte:

<https://marcelo.pimenta.com.br/abordagens-para-inovacao-qual-a-relacao-entre-design-thinking-metodos-ageis-lean-startup-e-design-sprint/>

Testes A/B

Testes A/B

- » Existe o cenário onde duas implementações de requisitos "competem" entre si
- » Exemplo: loja virtual
- » Sistema recomendação: quem compra P também compra X,Y,Z
 - Versão original
 - Nova versão, proposta por alguns devs
- » Vale a pena migrar para nova versão?
 - Testes A/B: deixar os dados decidirem

Testes A/B

- » Testes A/B **podem ser usados**, por exemplo, **quando se constrói um MVP** (com requisitos A) e, depois de um ciclo construir-medir-aprender pretende-se testar um novo MVP (com requisitos B).
- » Um outro cenário muito comum são testes A/B envolvendo **componentes de interfaces** com o usuário.
 - Exemplo: dados dois layouts da página de entrada de um site, um teste A/B pode ser usado para decidir qual resulta em maior engajamento por parte dos usuários.

Versões de controle e tratamento

- » Versão A: controle (sistema original, com os requisitos A)
- » Versão B: tratamento (sistema com novos requisitos B)
- » Novos usuários:

```
version = Math.Random(); // número aleatório entre 0 e 1
if (version < 0.5)
    "execute a versão de controle"
else
    "execute a versão de tratamento"
```

No final do experimento

- » Analisam-se os dados, isto é, alguma métrica
- » Exemplo: taxa de conversão de visitas em compras
- » Versão B introduz ganhos **estatisticamente** relevantes?
 - Sim, vamos migrar para ela
 - Não, continuamos com o algoritmo original

Calculadoras de tamanho de amostras

- » Os testes podem demandar um **número extremamente elevado de clientes**.
- » Informa-se:
 - Taxa de conversão atual (1%)
 - Ganho pretendido (10%)
- » Calculadora informa:
 - Tamanho da amostra necessário: 200K por versão
- » [Link para calculadora](#)

Calculadoras de tamanho de amostras

- » São usados por todas as grandes empresas da Internet
 - No **Facebook**, as inovações que os engenheiros implementam são imediatamente liberadas para uso por usuários reais.
 - Testes A/B são uma abordagem experimental para descobrir o que os clientes querem, a qual dispensa elicitar requisitos de forma antecipada e escrever especificações.
 - Testes A/B permitem detectar cenários nos quais os usuários começam a usar novas funcionalidades de modo inesperado.
 - Isso permite que os engenheiros aprendam com a diversidade de usuários e apreciem as diferentes visões que eles têm do Facebook.